University of Southampton

COMP6237 Data Mining
Lecture 4: Embedding Data (Dimensionality Reduction II)

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS

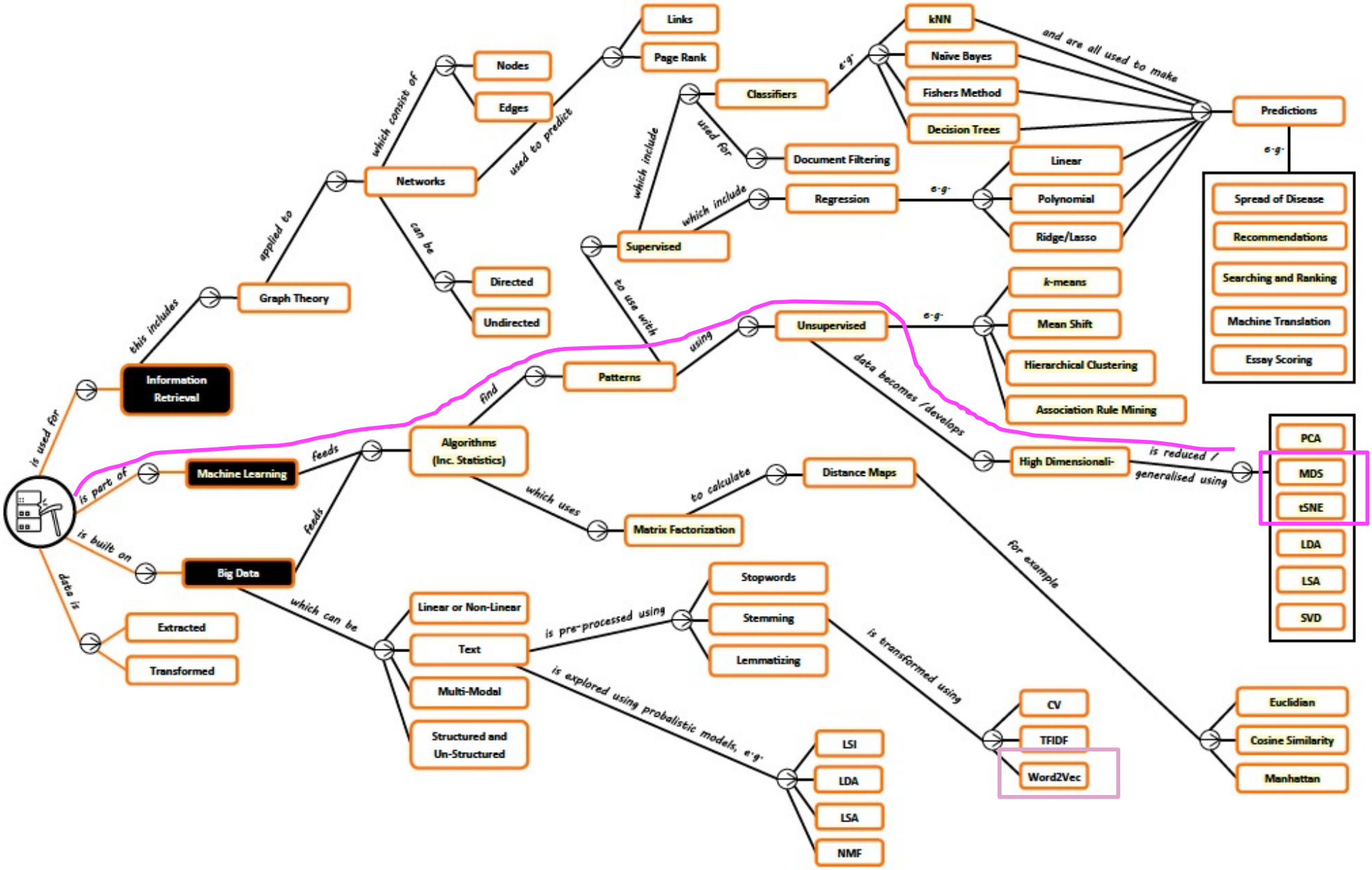
University of Southampton

Lecture slides available here:

http://comp6237.ecs.soton.ac.uk/zh.html

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Dimensionality Reduction II – **Roadmap**



Dimensionality Reduction II – Textbook

University of Southampton

CHAPTER 7 Dimensionality Reduction

We saw in Chapter 6 that high-dimensional data has some peculiar characteristics, some of which are counterintuitive. For example, in high dimensions the center of the space is devoid of points, with most of the points being scattered along the surface of the space or in the corners. There is also an apparent proliferation of orthogonal axes. As a consequence high-dimensional data can cause problems for data mining and analysis, although in some cases high-dimensionality can help, for example, for nonlinear classification. Nevertheless, it is important to check whether the dimensionality can be reduced while preserving the essential properties of the full data matrix. This can aid data visualization as well as data mining. In this chapter we study methods that allow us to obtain optimal lower-dimensional projections of the data.

- Data Mining and Machine Learning: Fundamental Concepts and Algorithms M. L. Zaki and W. Meira

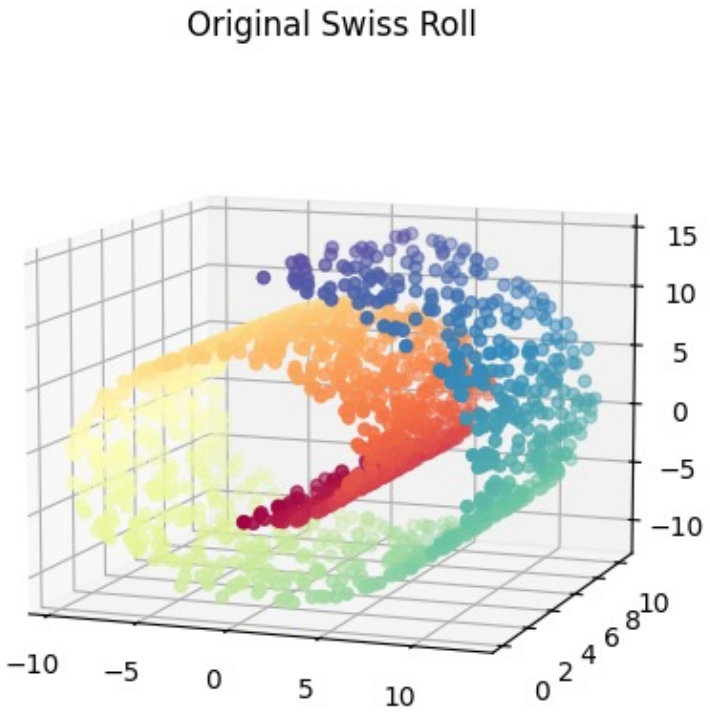
Chapter 11

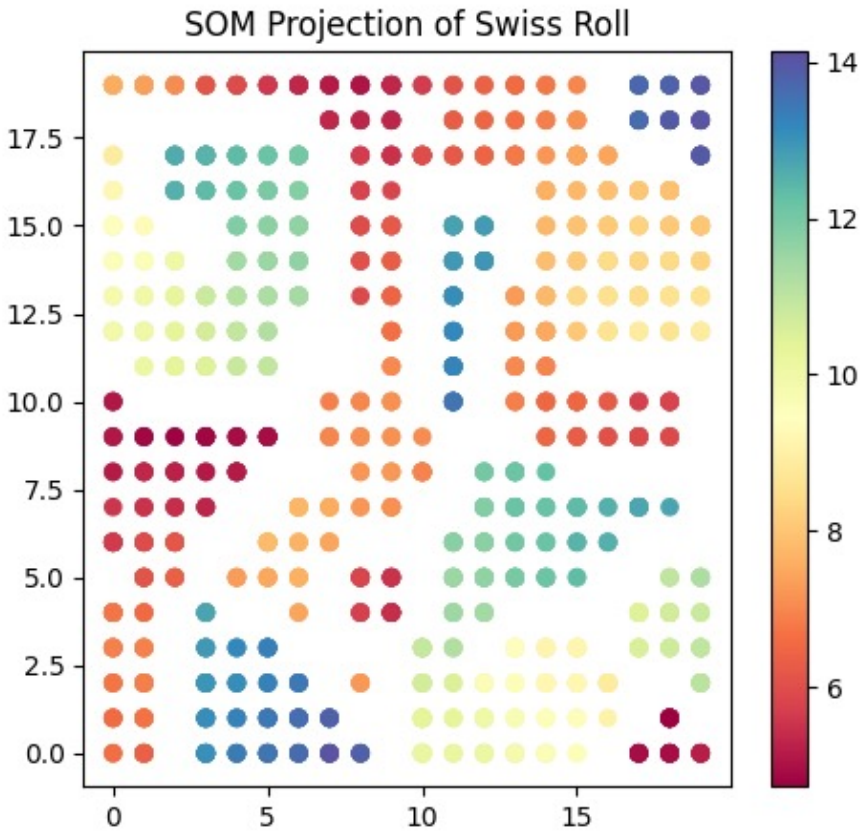
Dimensionality Reduction

There are many sources of data that can be viewed as a large matrix. We saw in Chapter 5 how the Web can be represented as a transition matrix. In Chapter 9, the utility matrix was a point of focus. And in Chapter 10 we

- Mining of Massive Datasets J. Leskovec *et al*

Dimensionality Reduction II – **Overview (1/3)**

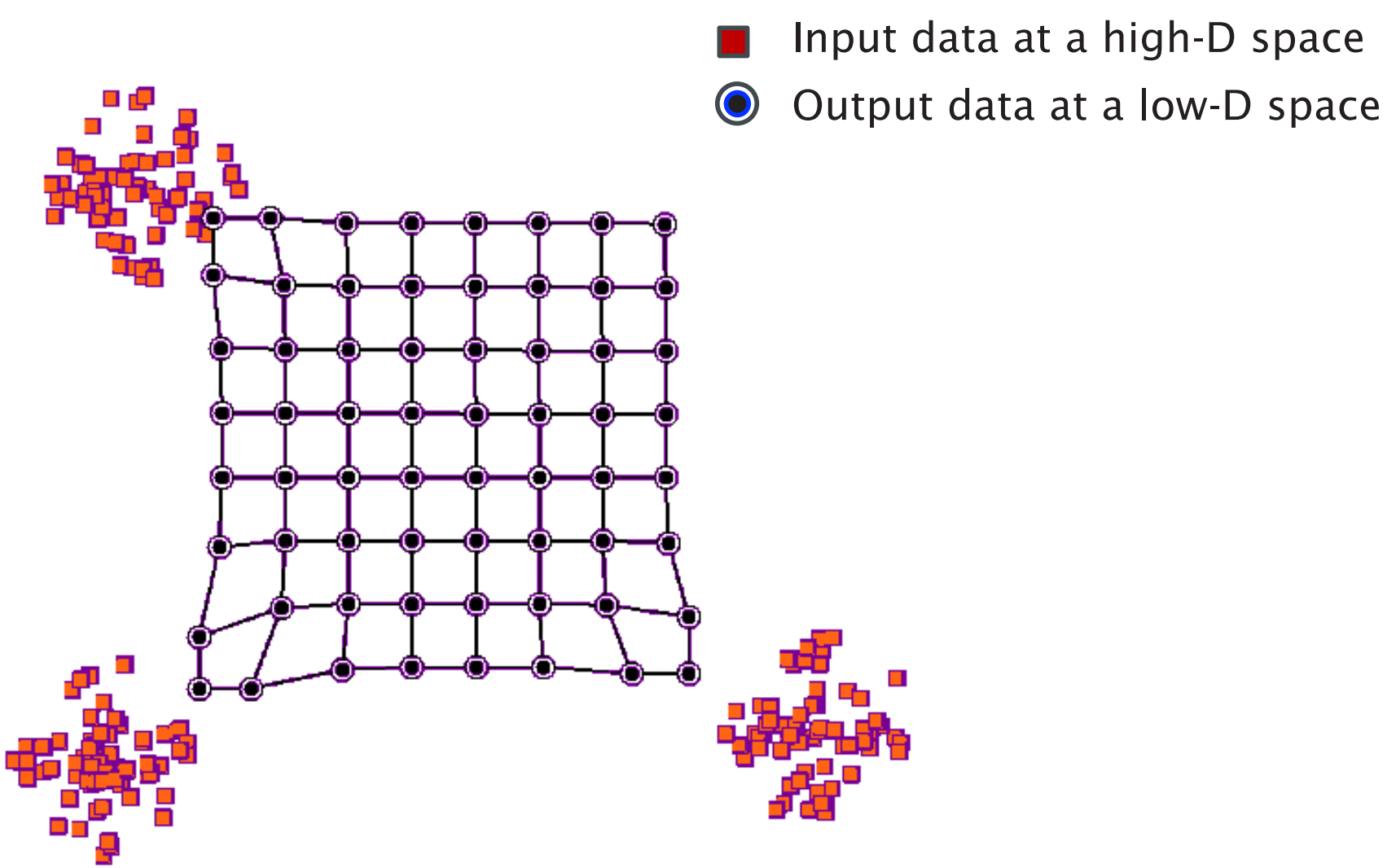




Key idea: maintain the topological structure of the original data!

Dimensionality Reduction II – **Overview (2/3)**

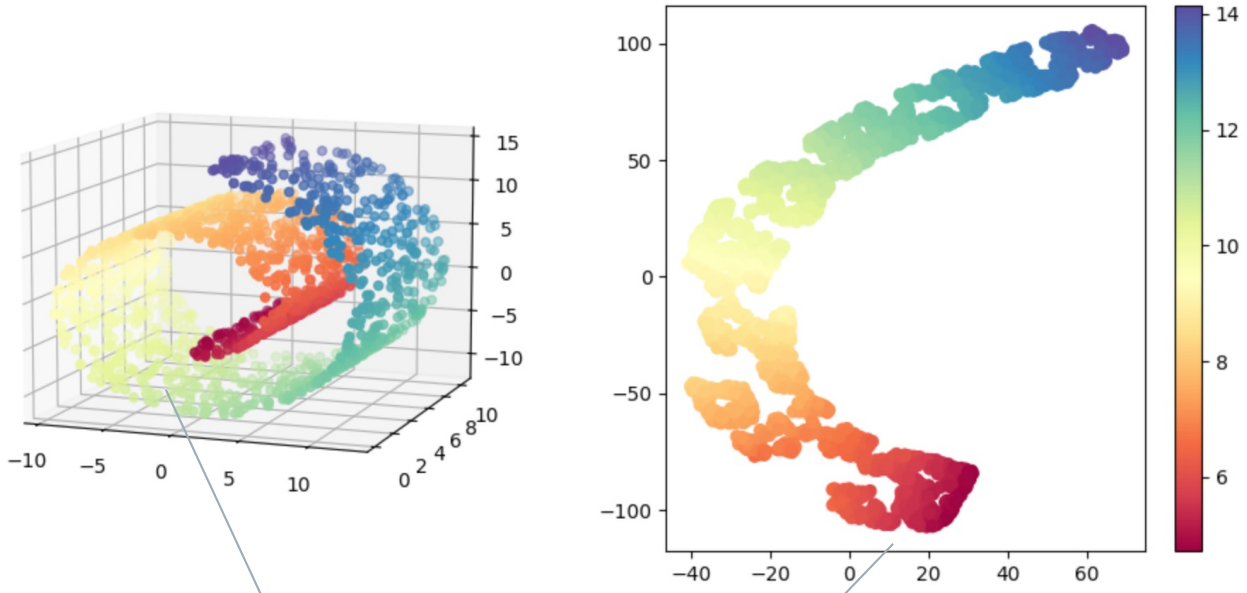
Method I: **Self-Organising Maps (SOM)**: *distance-based method*

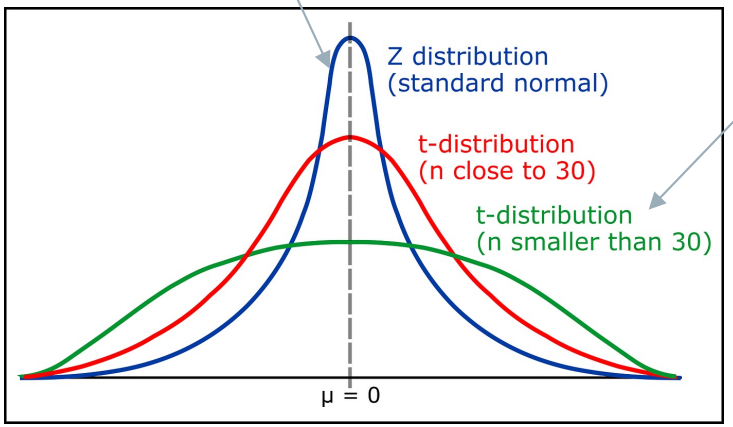


Dimensionality Reduction II – **Overview (3/3)**

Method II/III: **Stochastic Neighbour Embedding (SNE) & t-SNE**

Distribution-based method





Dimensionality Reduction II – **Learning Outcomes**

- **LO1**: understand the basic idea of all the learned dimensionality reduction methods (exam)
  - E.g., describe the basic idea of the suggested algorithm
  - E.g., understand the pros and cons of the learned algorithms
  - E.g., given a dataset, be prepared to follow the selected algorithm to calculate its key steps on the given data
- **LO2**: Implement the learned algorithms for dimensionality reduction (course work)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf*
- *ChatGPT or other AI-based techs*

Embedding Data

Understanding large data sets is *hard*

Especially when the data are highly dimensional

It would help if we knew:

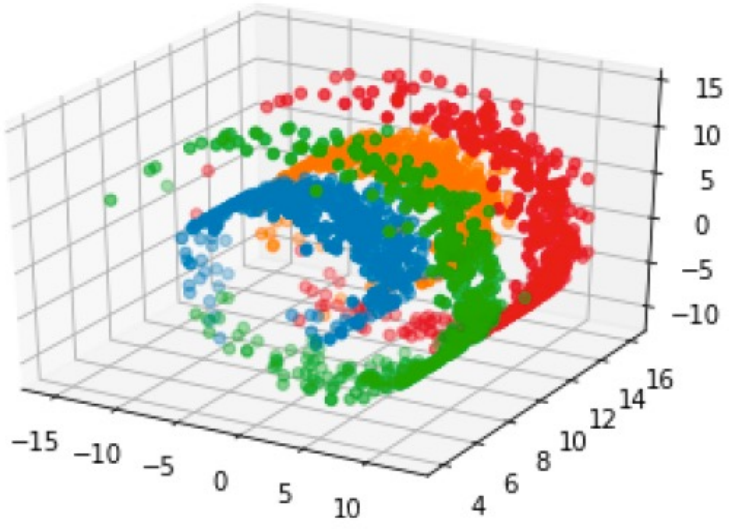
- which data items are similar
- which features are similar

Embedding Data

With 2D data, we can plot it to easily visualise relationships
This is not possible with highly dimensional data
However: PCA can reduce the dimensionality to 2, based on the first and second principle axes

Embedding Data - PCA

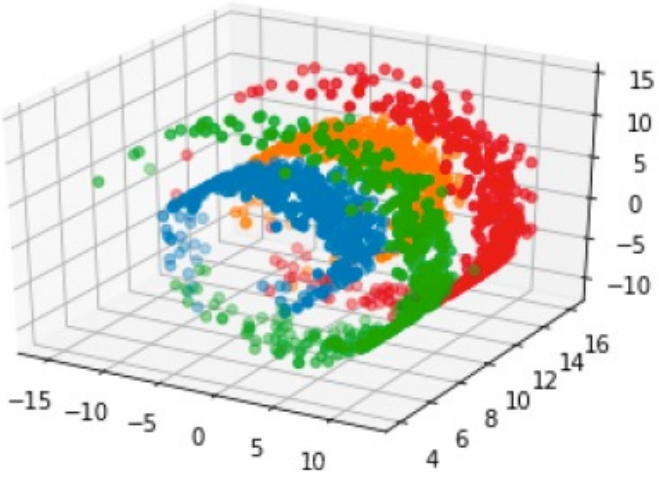
We can use the so called 'Swiss Roll' data set to exemplify this:

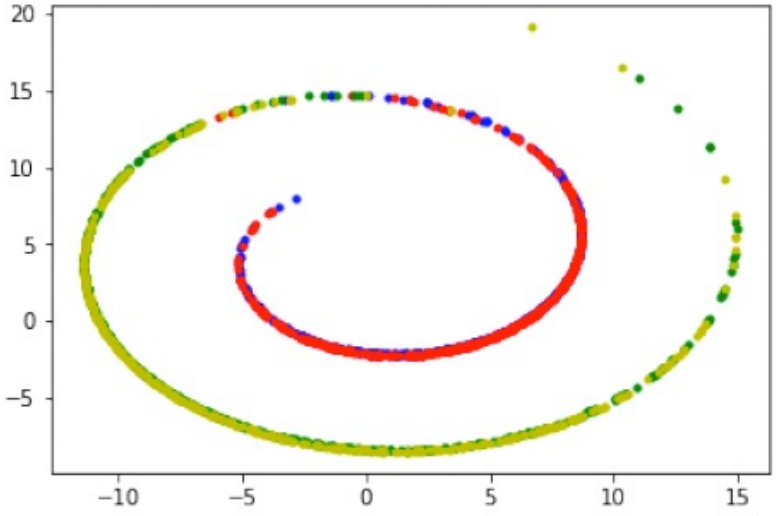


The data in 3D has 4 clearly separate groups

Embedding Data - PCA

We can use the so called 'Swiss Roll' data set to exemplify this:





The data in 3D has 4 clearly separate groups

Using PCA, it does not separate the data well at all

Embedding Data - PCA

Unfortunately there is no control over the distance measure

Using axes of greatest variance does not mean similar things appear close together.

PCA is only rotation of original space, followed by removal of less significant dimensions

Embedding Data – Self-Organising Maps

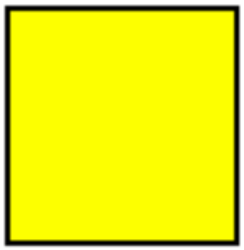
University of Southampton

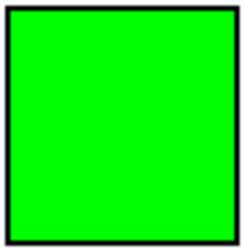
Kohonen 1982: Self-Organising Maps (SOM)

- Inspired by neural networks
- 2D n by m array of nodes
- Units close to each other are considered to be neighbours
- Maps high dimensional vectors to unit with coordinates closest (Euclidean Distance)
- This is the *best matching unit*

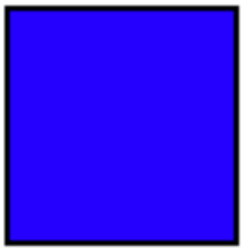
Embedding Data – Self-Organising Maps

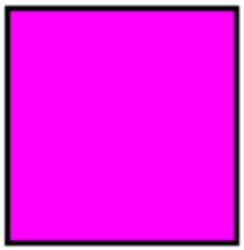

R = 255
G = 0
B = 0

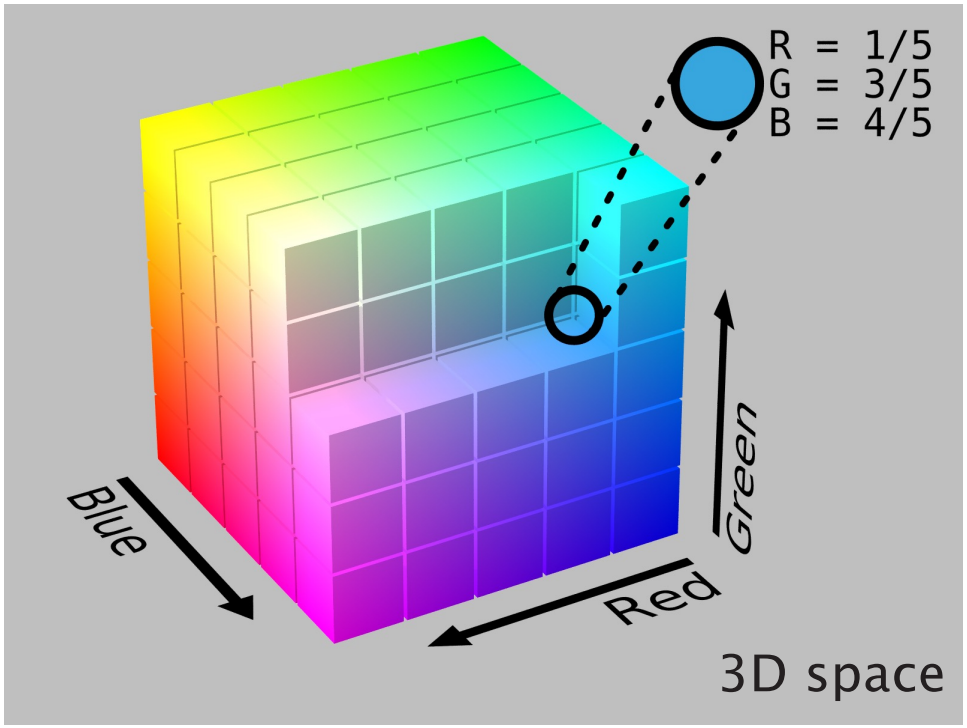

R = 255
G = 255
B = 0


R = 0
G = 255
B = 0


R = 0
G = 255
B = 255


R = 0
G = 0
B = 255


R = 255
G = 0
B = 255



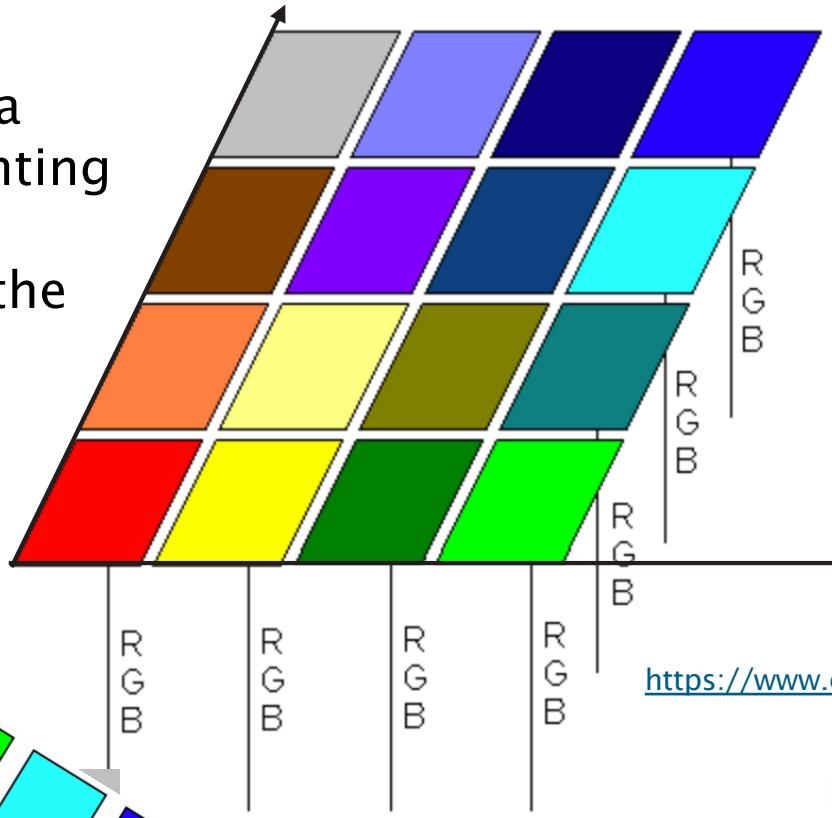
https://en.wikipedia.org

A good mapping would group the red, green, and blue colors far away from one another and place the intermediate colors between their base colors (e.g. yellow close to red and green, teal close to green and blue, etc)

https://www.cs.hmc.edu/~kpang/nn/som.html

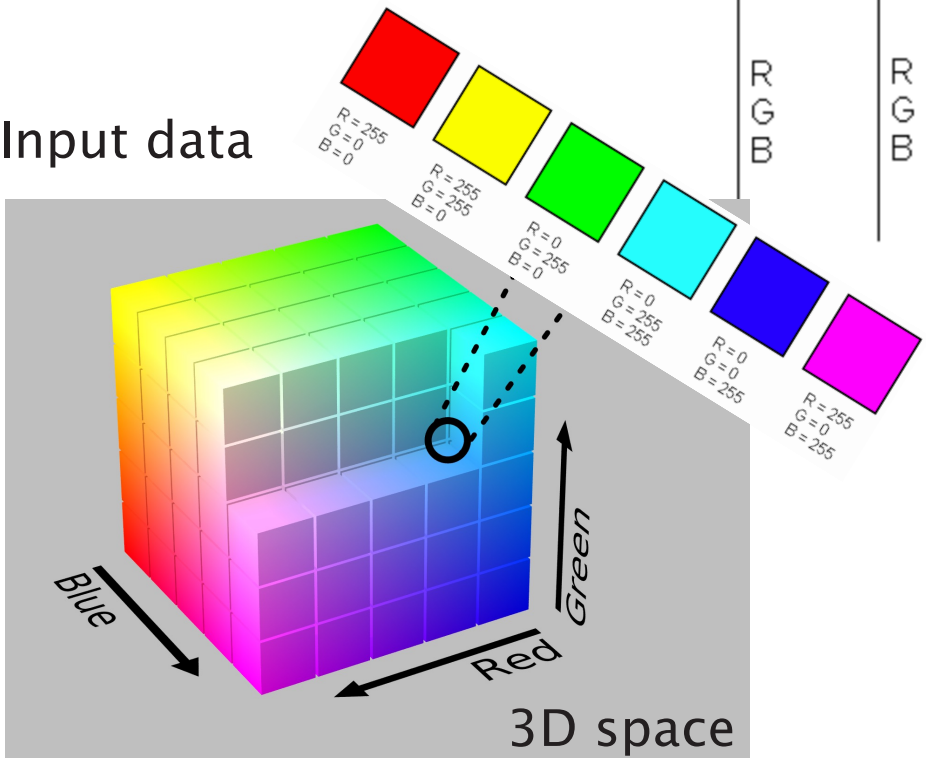
Embedding Data – Self-Organising Maps

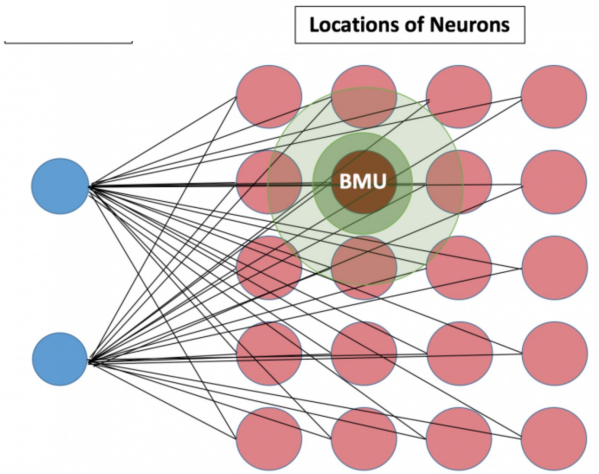
Each neuron contains a weight vector representing its RGB values and a geometric location in the grid.



https://www.cs.hmc.edu/~kpang/nn/som.html

Input data





https://wonikjang.github.io

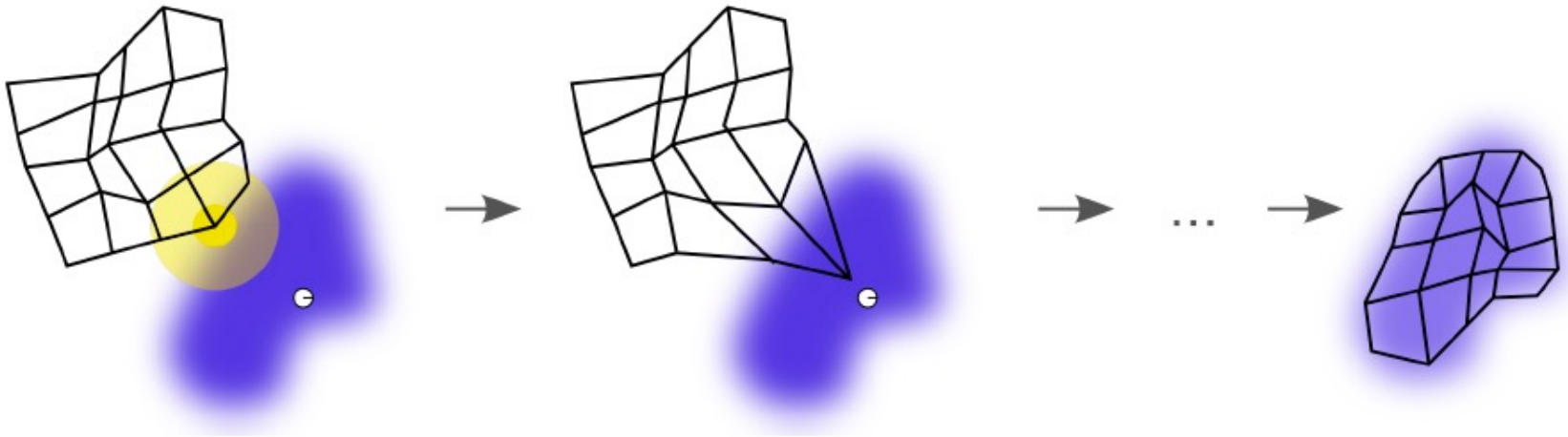
Embedding Data – Self-Organising Maps

SOMs: two phases

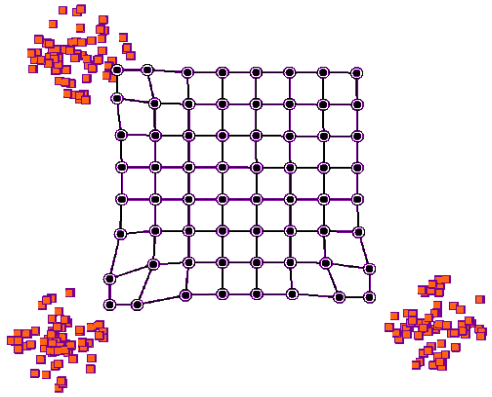
- training
- mapping

To start training, the set of nodes each has a random starting position defined in the feature space.

This is then updated by taking one feature vector, finding which unit is the *best matching unit* (BMU) then moving that unit and, to a lesser extent, its neighbours, closer to that data point



Embedding Data – Self-Organising Maps



Credit: Chompinha

Algorithm 1: Training Self-Organising Maps

Data: N data points with d dimensional feature vectors X_i $i = 1 \ldots N$, number of iterations λ

$\boldsymbol{w}$ = randomly initialise n xm units with weight vector ;

t = 0;

while $t < \lambda$ **do**

  for *each* x_i **do**

    $BMU = w_{nm}$ with min distance;

    Update BMU and its neighbours by moving closer to x_i;

  end

  $t = t + 1$

end

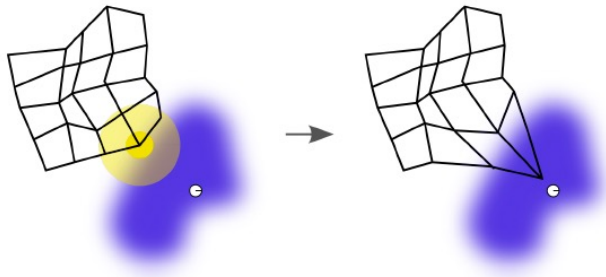
Embedding Data – Self-Organising Maps

To update the weight vector $\boldsymbol{w}$

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) + \theta(u, v, t)\alpha(t)(x_i - \boldsymbol{w}(t))$$

where θ is the neighbourhood weighting function (usually Gaussian)

e.g., $\theta = \begin{cases} 1 & \text{if the node is the winner} \\ 0.5 & \text{if the node is immediate neighbour of the winner} \\ 0 & \text{otherwise} \end{cases}$



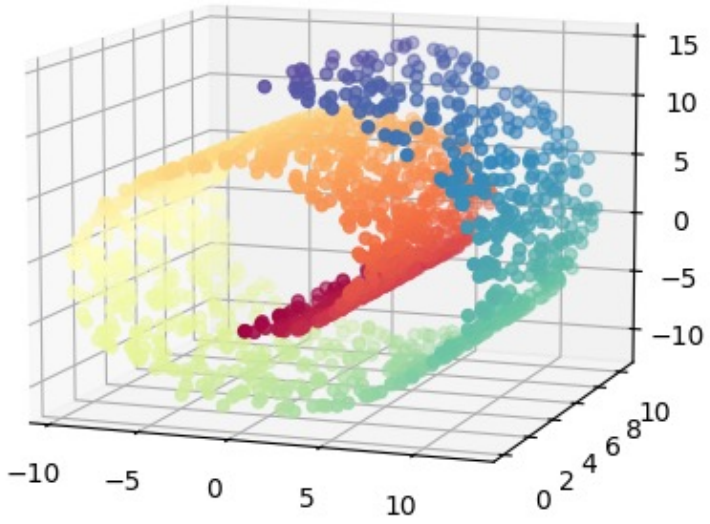
α is the learning rate

x_i is the input vector

u is the winner node, and v is the node whose weight is $\boldsymbol{w}$

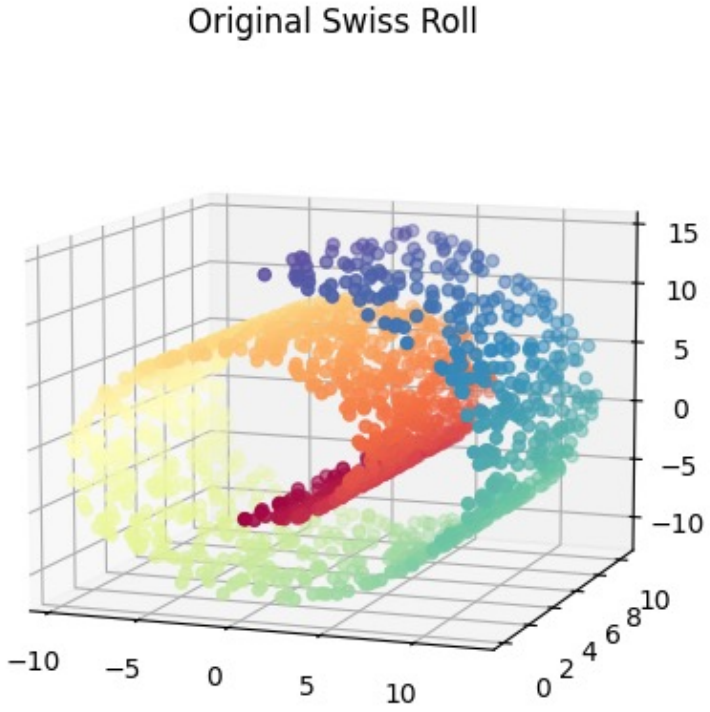
Both the learning rate and the neighbourhood weighting function get smaller over time

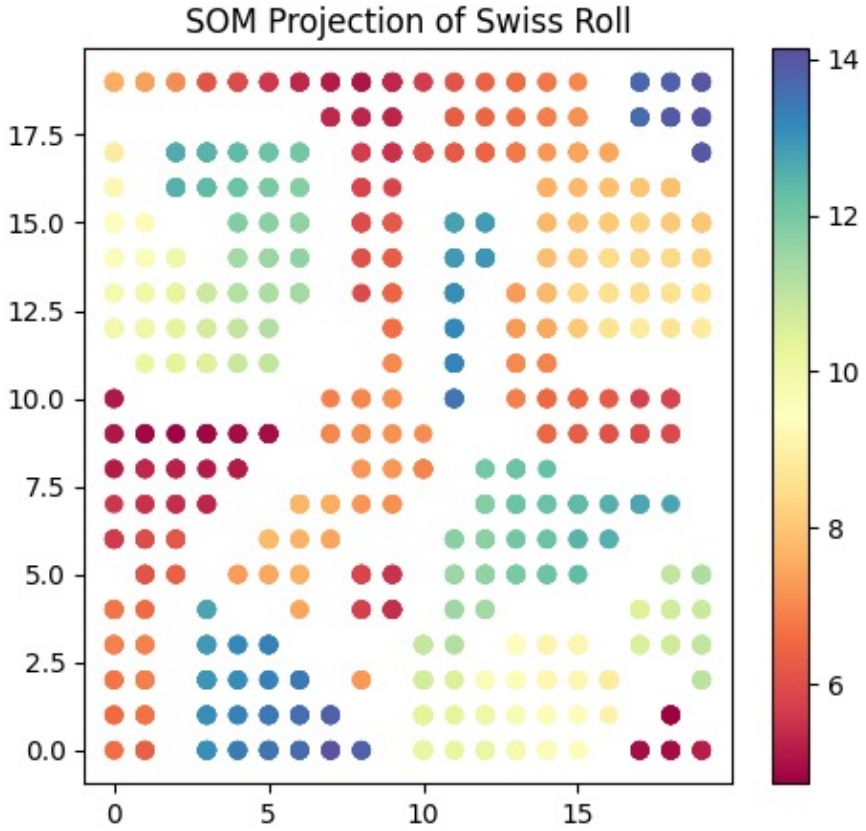
Embedding Data – Self-Organising Maps



The data in 3D has a clear structure

Embedding Data – Self-Organising Maps





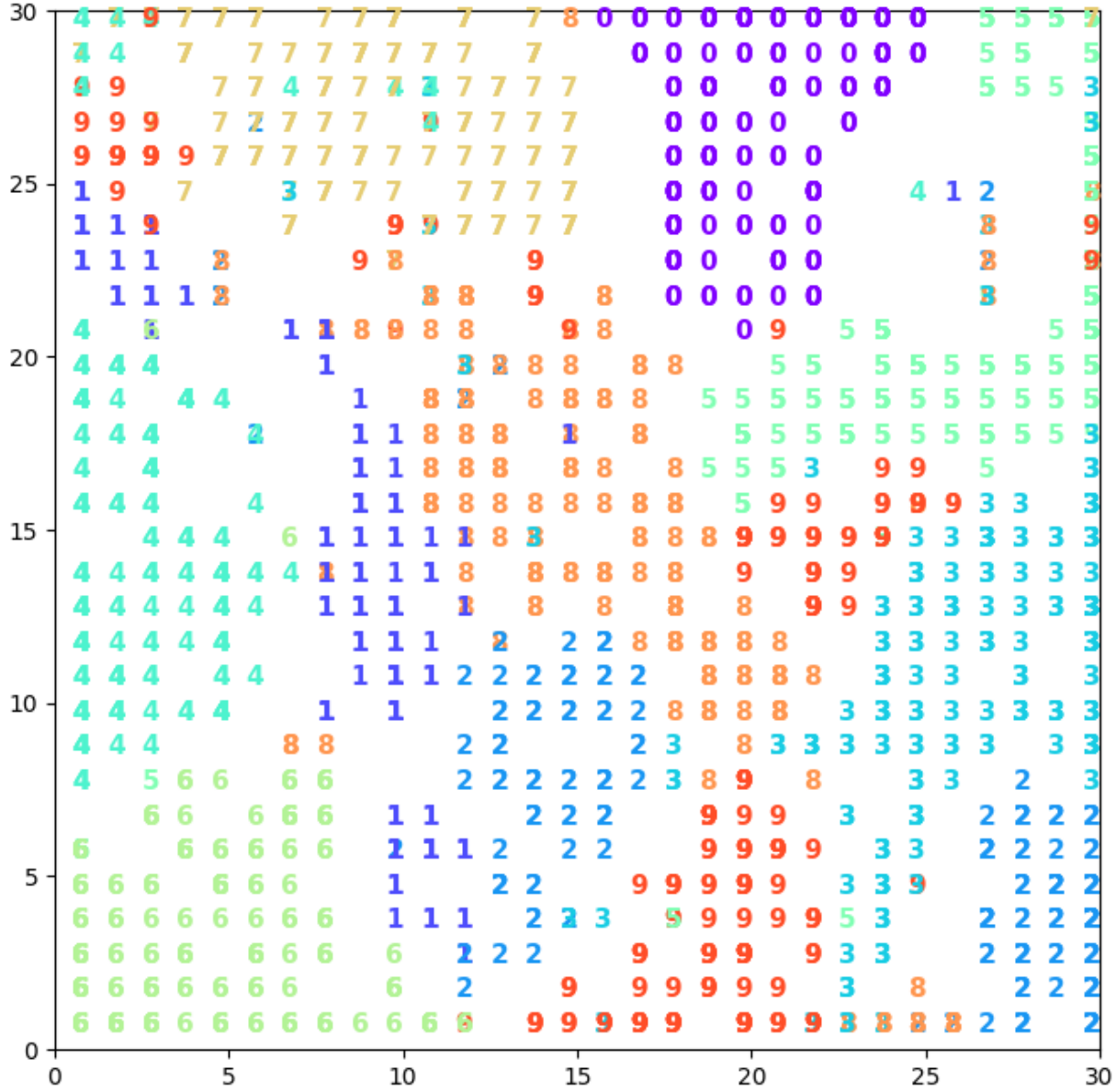
The data in 3D has a clear structure

Using SOM, it can much better group the data

ipynb mean centering demo https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19

Embedding Data – Self-Organising Maps

With the MNIST digits, the results are quite impressive



ipynb mean centering demo

https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19

Embedding Data – Stochastic Neighbour Embedding

Stochastic Neighbour Embedding (SNE)
Works in a similar way to MDS
MDS optimises distances, SNE optimises the distribution of data
Aims to make the distribution of the projected data in low dimensional space close to the actual distribution in high dimensional space

Embedding Data – Stochastic Neighbour Embedding

To calculate the source distribution:

We define a conditional probability that high-dimensional x_i would pick x_j as a neighbour if the neighbours were picked in proportion to their probability density under a Gaussian centred at x_i

$$p_{j|i} = \frac{exp^{-||x_i - x_j||^2/2\sigma_i^2}}{\sum_{k \neq i} exp^{-||x_i - x_k||^2/2\sigma_i^2}}$$

The SNE algorithm chooses σ for each data point such that smaller σ is chosen for points in dense parts of the space, and larger σ is chosen for points in sparse parts

Embedding Data – Stochastic Neighbour Embedding

To calculate the target distribution:

Define a conditional probability that low-dimensional y_i would pick y_j as a neighbour if the neighbours were picked in proportion to their probability density under a Gaussian centred at y_i

$$q_{j|i} = \frac{exp(-||y_i - y_j||^2)}{\sum_{k \neq i} exp(-||y_i - y_k||^2)}$$

In this space we assume the variance of all Gaussians is $1/\sqrt{2}$ in this space

Embedding Data – Stochastic Neighbour Embedding

To measure the difference between two probability distributions we use the *Kullback-Leibler* (KL) Divergence:

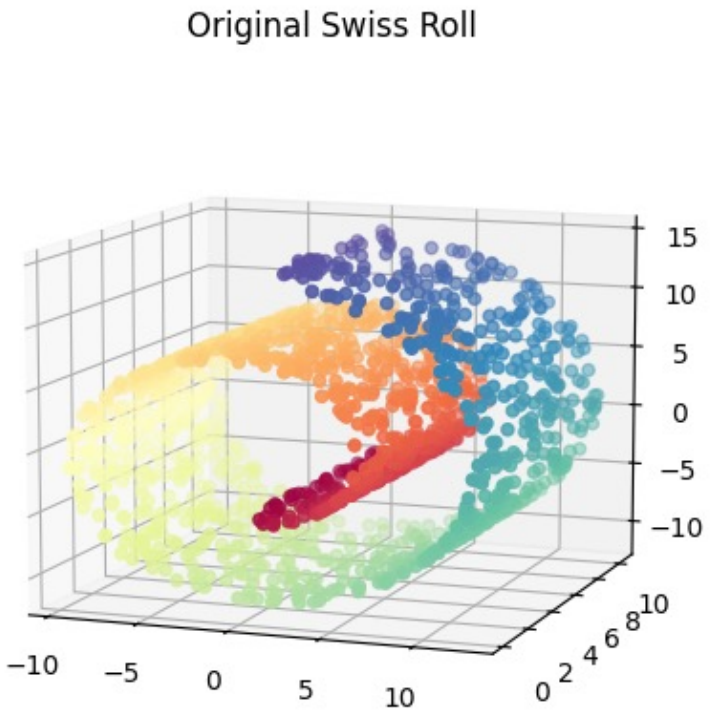
$$D_{KL}(P|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

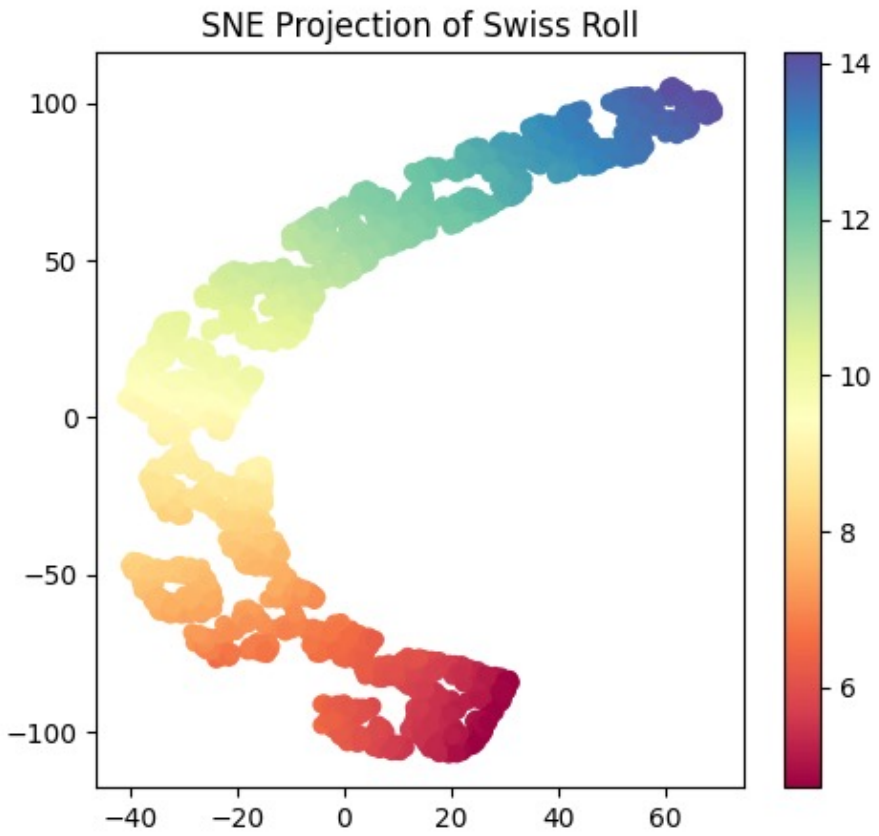
The cost function is the KL divergence summed over all data points

$$C = \sum_i \sum_j p_{i|j} \log \frac{p_{j|i}}{q_{j|i}}$$

C can be minimised using *gradient descent* - but..
difficult to optimise, leads to crowded visualisations, big clumps of data together in the center

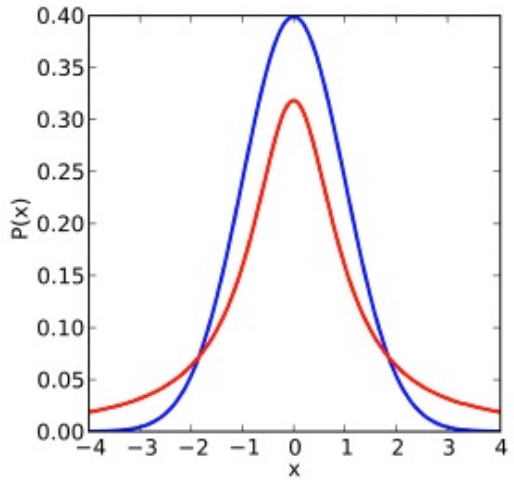
Embedding Data – Stochastic Neighbour Embedding





Embedding Data – t- Stochastic Neighbour Embedding

In 2008 Maaten and Hinton came up with a way to improve SNE, by replacing the Gaussian distribution for the lower dimensional space with a Student's t distribution.



Student's t distribution in red, Gaussian distribution in blue

The Student's t distribution has a much longer tail, helps avoid clumping in the middle.

Embedding Data – t- Stochastic Neighbour Embedding

The cost function is also modified, making gradients simpler, so faster to compute

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

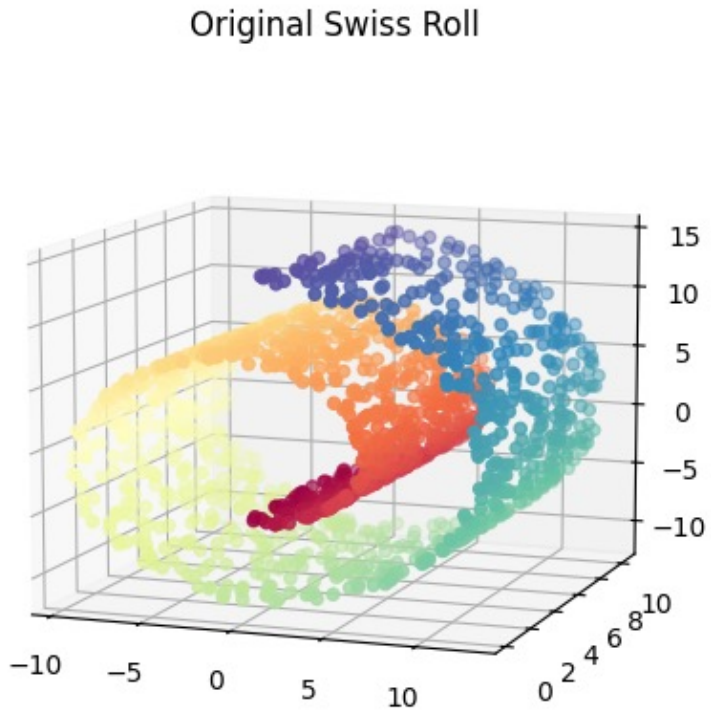
To alleviate crowding using Student's t distribution in the lower dimensional space:

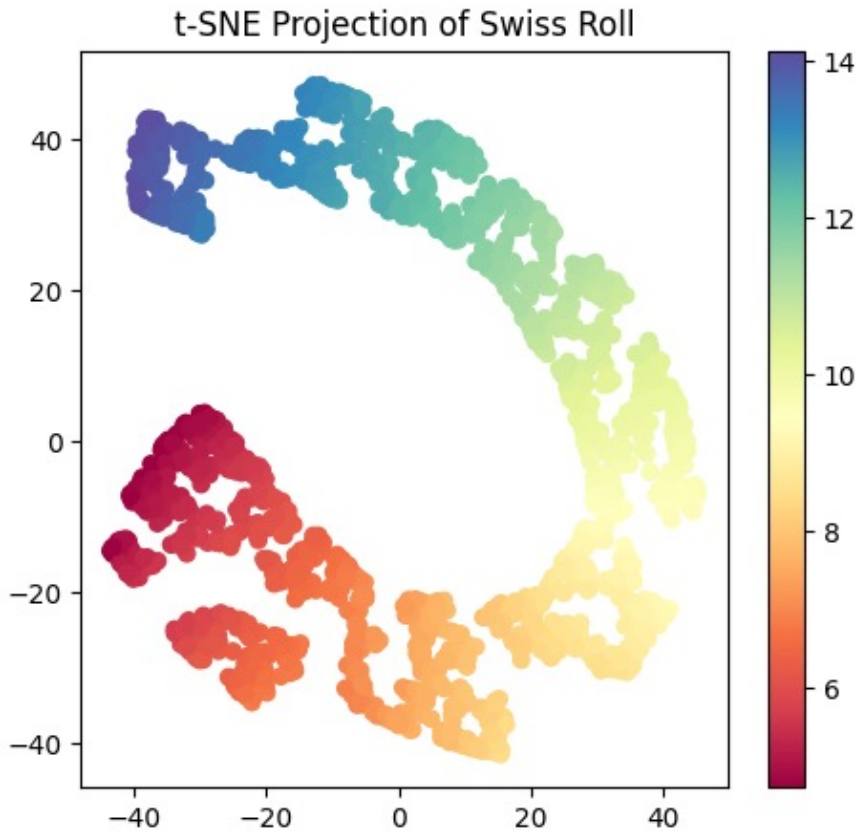
$$q_{ij} = \frac{(1 + ||x_i - x_j||)^{-1}}{\sum_{k \neq i}(1 + ||x_i - x_k||)^{-1}}$$

we use 1 degree of freedom with the Student's t distribution, equivalent to a Cauchy distribution

Embedding Data – t- Stochastic Neighbour Embedding

For the Swiss Roll data:





ipynb mean centering demo https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19

Embedding Data – Summary

	Key Idea	Pros	Cons
SOM	Uses **competitive learning** to map high-dimensional data onto a **grid of neurons** (usually 2D) while preserving topology.	- Good for **clustering** and **visualization** - Preserves **topological relationships** - Works well with **unsupervised learning**	- Can be **slow** for large datasets - **Grid size** affects performance - Not optimal for non-topological structures
SNE	Converts high-dimensional distances into **probabilities** and minimizes **KL divergence** to project points into a low-dimensional space.	- Preserves **local structure** well - Useful for **visualization**	- Prone to **crowding problem** (points collapse) - **Difficult to optimize**
t-SNE	Improves SNE by using a **Student's t-distribution** in the low-dimensional space, avoiding crowding and spreading points better.	- **Better clustering** than SNE - Preserves **both local & some global structures** - **Widely used** for data visualization	- **Computationally expensive** for large datasets - Results **vary** based on hyperparameters (e.g., perplexity) - Hard to interpret distances globally

- **SOM** is great for clustering but limited in flexibility.
- **SNE** captures local structures but suffers from the crowding problem.
- **t-SNE** improves SNE and is one of the most used techniques for **high-dimensional visualization**.

Appendix – Word2Vec

Instead of projecting high dimensional data down in to 2 or 3 dimensions, we can use a medium dimensionality, keeping the useful information, capturing the key distinguishing features
This is called an *embedding*
For example: word2vec

Appendix – One Hot Encoding

For example: Documents

We use a 'Bag of Words', where each word is a vector:

- a $\rightarrow [1, 0, 0, 0, 0, 0, 0, 0, \ldots, 0]$
- aa $\rightarrow [0, 1, 0, 0, 0, 0, 0, 0, \ldots, 0]$
- aardvark $\rightarrow [0, 0, 1, 0, 0, 0, 0, 0, \ldots, 0]$
- aardwolf $\rightarrow [0, 0, 0, 1, 0, 0, 0, 0, \ldots, 0]$

This is called *One Hot Encoding*

Appendix – One Hot Encoding

University of Southampton

What problems does this encoding have?

- all vectors are *orthogonal*, i.e. unrelated

But we know that many words in English are related, e.g. 'rain', 'drizzle', 'downpour', 'shower', 'squall' all mean pretty much the same thing.

- vectors are very long and *very* sparse

English has over 250,000 words (depending on how you count them) so each vector that could fully describe a document should be 250,000 long for every word.

Appendix – word2vec

Boiling this data down to a manageable vector size, while still retaining meaning is not a simple task

word2vec was proposed in 2013 by Mikolov *et al* (although the paper was rejected by the ICLR conference!)

Published at NeurIPS 2013 and cited over 40,000 times, the work introduced the seminal word embedding technique word2vec. Demonstrating the power of learning from large amounts of unstructured text, the work catalyzed progress that marked the beginning of a new era in natural language processing.

NeurIPS 2023 'Test of Time Award'

https://blog.neurips.cc/2023/12/11/announcing-the-neurips-2023-paper-awards/

Appendix – word2vec

University of Southampton

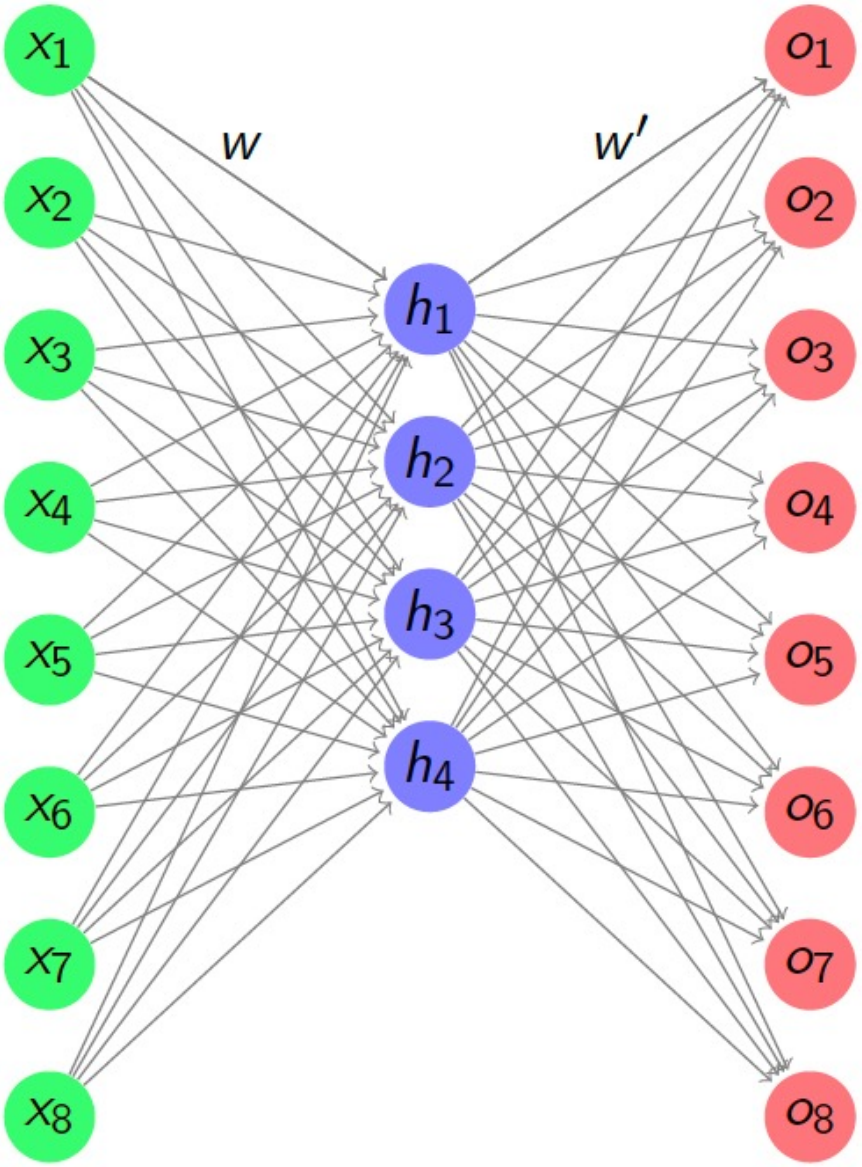
For example:
"the quick brown fox jumps over the lazy dog"

The word 'brown' has the words 'the', 'quick', 'fox' and 'jumps' close by
This gives training samples:

- 'the', 'brown'
- 'quick', 'brown'
- 'fox', 'brown'
- 'jumps', 'brown'

which we train a simple neural network with.

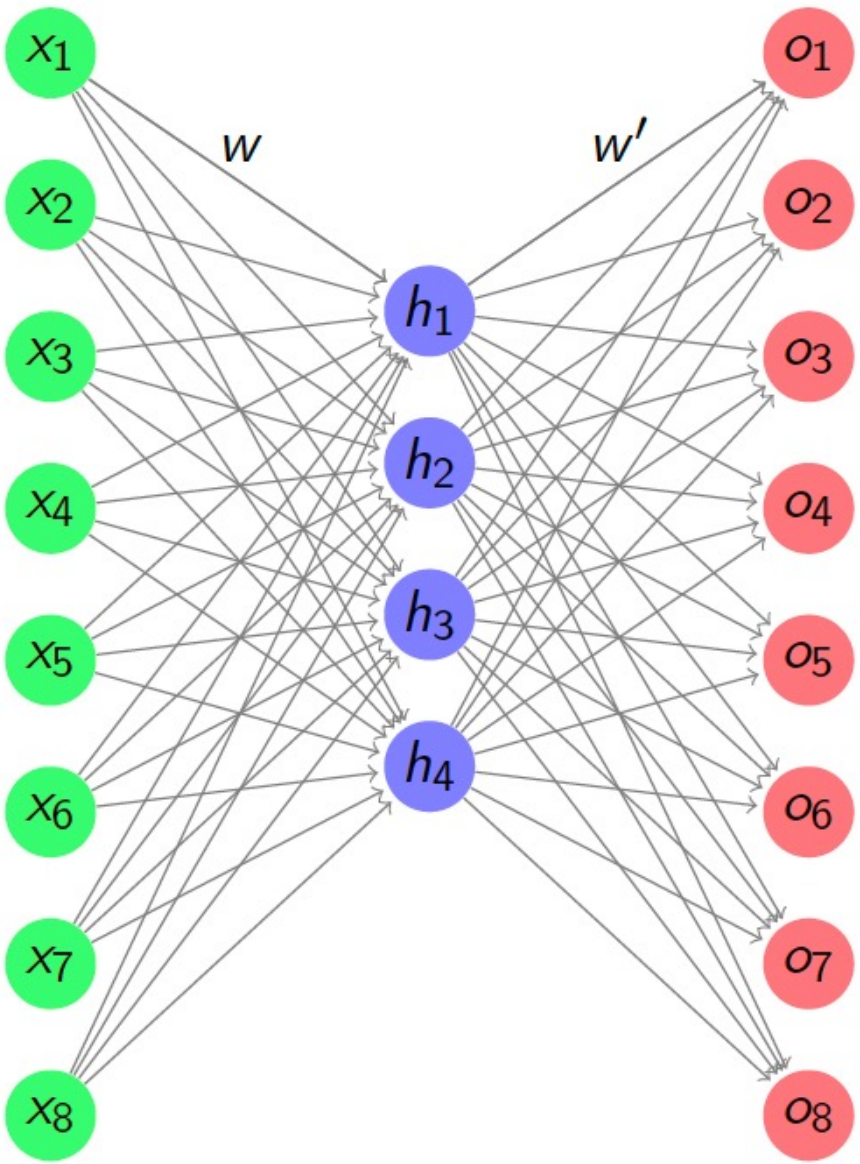
Appendix – word2vec



input vector is the 'one hot' encoding of the word in question

Credit: Jo Grundy

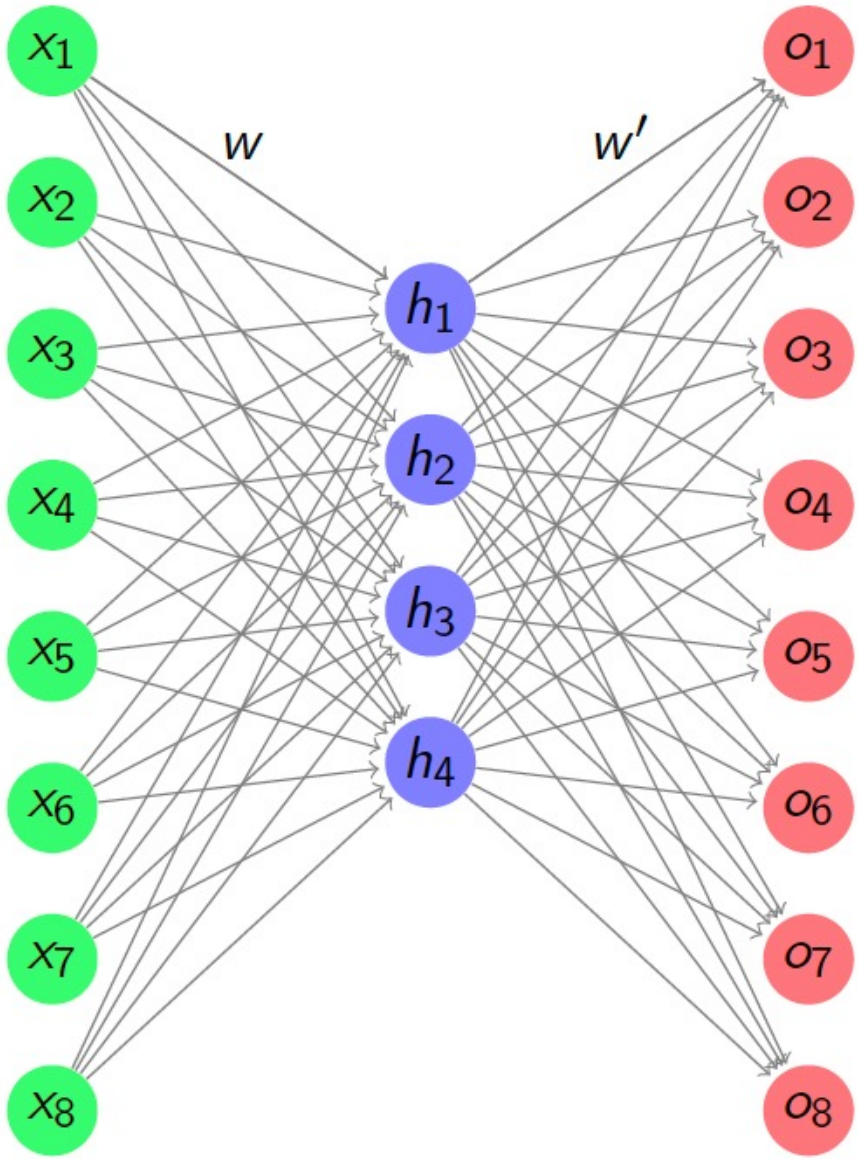
Appendix – word2vec



input vector is the 'one hot' encoding of the word in question

the output vector is the vector for the predicted word.

Appendix – word2vec

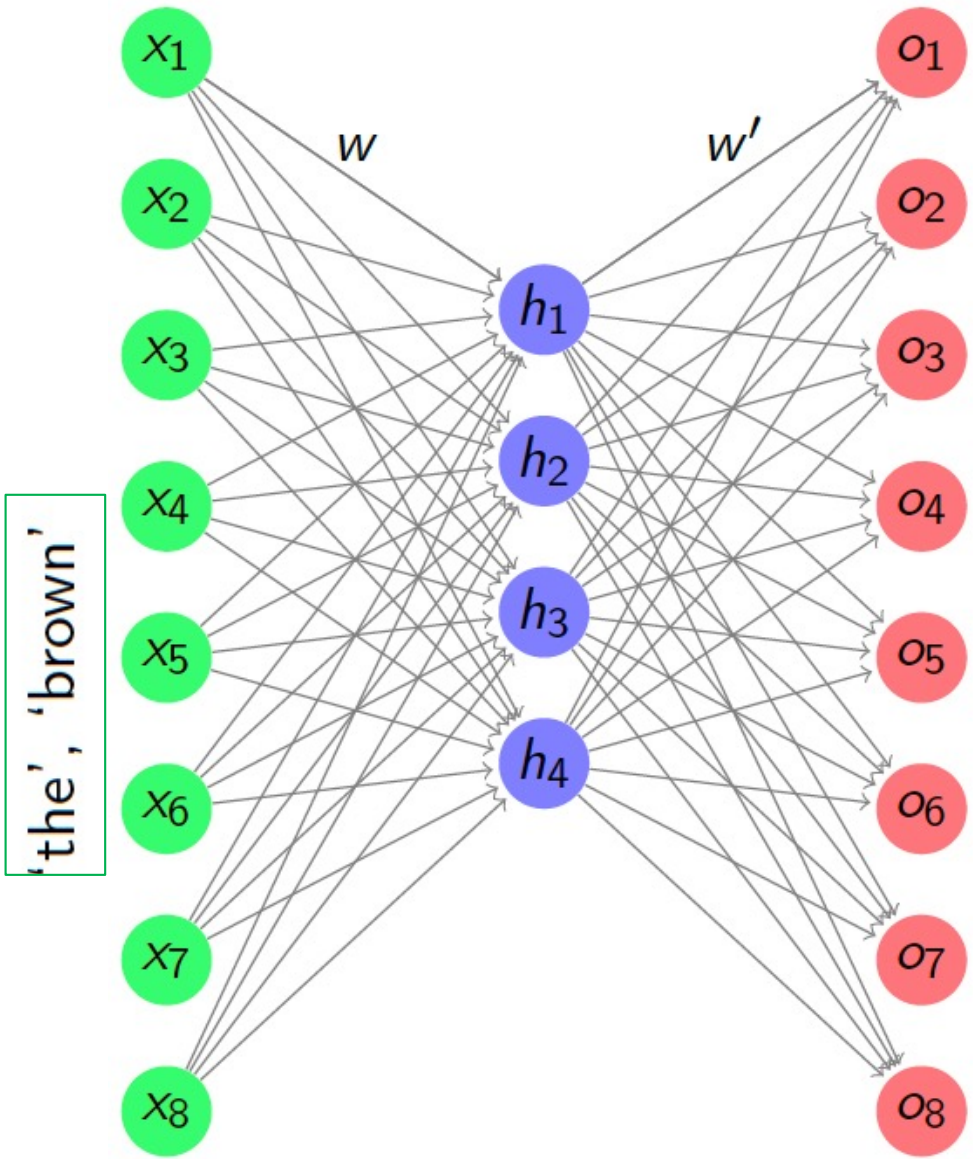


input vector is the 'one hot' encoding of the word in question

the output vector is the vector for the predicted word.

the hidden layer learns a lower dimensional encoding of each word

Appendix – word2vec



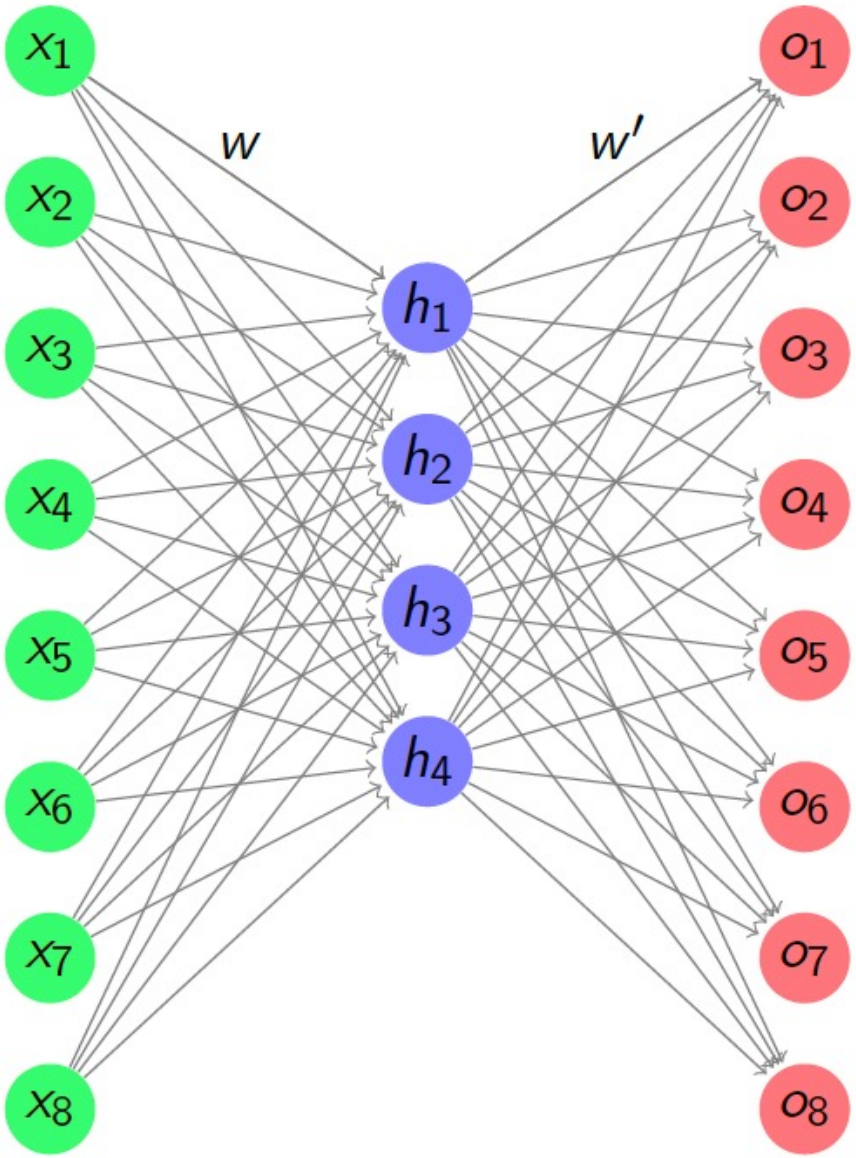
input vector is the 'one hot' encoding of the word in question

the output vector is the vector for the predicted word.

the hidden layer learns a lower dimensional encoding of each word

in training the correct word is used as the teaching signal, and back propagation is used to learn the weights to the hidden layer.

Appendix – word2vec



input vector is the 'one hot' encoding of the word in question

the output vector is the vector for the predicted word.

the hidden layer learns a lower dimensional encoding of each word

in training the correct word is used as the teaching signal, and back propagation is used to learn the weights to the hidden layer.

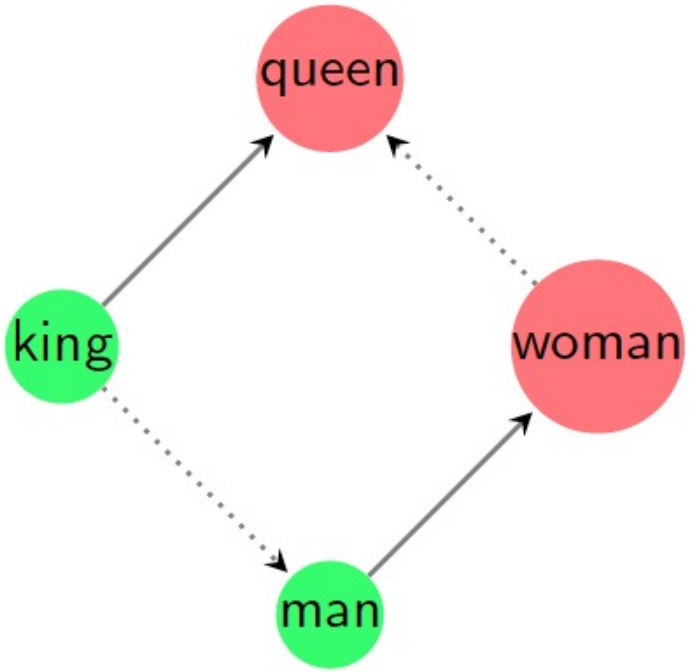
the output from the hidden layer after training is used as the lower dimensional representation

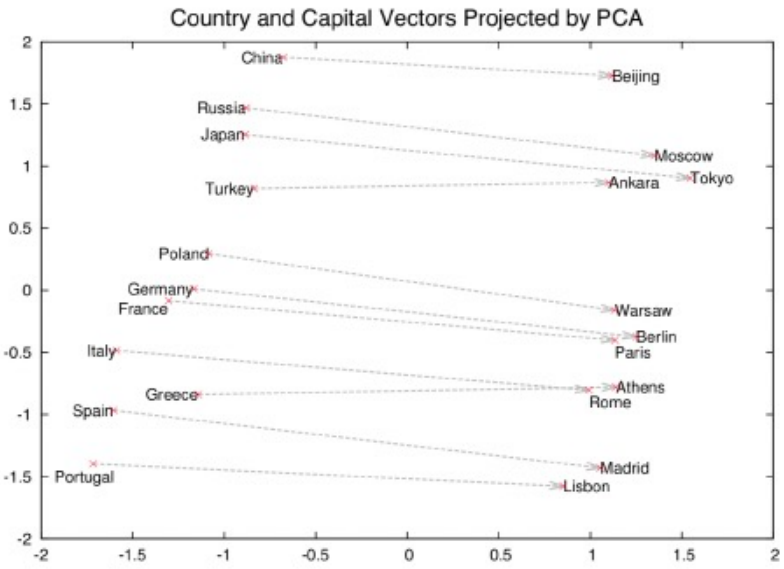
Credit: Jo Grundy

Appendix – word2vec

These lower dimensional representations can include a good deal of semantic meaning

i.e. vector(king) - vector(man) + vector(woman) ≈ vector(queen)





From Mikolov *et al* NIPS 2013

Appendix – word2vec

Dimensionality reduction and visualisation is key to understanding the data

Useful for your coursework

There are many ways to do this, with the `sklearn.manifold` library:

https://scikit-learn.org/stable/modules/manifold.html